Check the Tree: Finding Wrong Results in PostgreSQL with Typed Derivation Trees and Commit-Directed Grammar Extension

Venu Kummari
v.kummari@iitg.ac.in
Indian Institute of Technology Guwahati
Guwahati, India

Bhuvan Chilukuti
c.bhuvan@iitg.ac.in
Indian Institute of Technology Guwahati
Guwahati, India

Gowtham Reddy
r.doma@iitg.ac.in
Indian Institute of Technology Guwahati
Guwahati, India

ABSTRACT

When a database system returns a wrong result, nothing tells the user: there is no error, no crash and nothing in the logs. Most fuzzers for PostgreSQL look for crashes, and the oracles that can detect wrong results are tied to small query generators of their own. As an example, PostgreSQL 17, 18 and master answer `true` for `'(,)'::int4range @> NULL`, although a strict operator must return NULL. We also found that of 23 recent query-processing fixes in PostgreSQL master, 20 change what a query returns, but only 9 of them crash or raise an internal error. This paper presents Arbor, a fuzzer that keeps every generated query as a typed derivation tree. Because the tree is kept, metamorphic oracles such as TLP, NoREC, expression rewrites, CTE and table fences and a row-level security check can rewrite a single node of known type at any level of the query. Triage then reduces a failure on the same tree. Plan variants and the last release of PostgreSQL act as two more oracles. In addition, an LLM reads recent commits and proposes grammar alternatives and equivalence rewrites, and a validation gate accepts them only if the server runs them, they reach the changed lines and no rewrite changes a result. In five one-hour trials per configuration, the typed grammar covers 36.7% of backend lines against 22.2% for random mutation, and the oracles double the number of plan shapes reached. We reported ten bugs found with Arbor; eight are fixed upstream, six of them with our patches, and Arbor rediscovers all ten in one 150-second run. Of 93 triaged candidates, 8 are real and 15 are duplicates of real bugs, and the false positives come from a few classes that we fixed. The LLM adds SQL features but no coverage, and one rewrite it proposed passed our gate although it is not an identity. On a benchmark of 23 real bugs, the best configuration found 2 of 20. Our main lesson is that oracle precision, not generator reach, is what limits a wrong-result fuzzer. Finally, with per-engine dialect overlays and optimizer-rule feedback, Arbor also tests DuckDB, SQLite, Turso and DataFusion, where it found five more bugs that we reported upstream with fixes.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Data management systems.*

KEYWORDS

fuzzing, database testing, test oracles, metamorphic testing, grammar-based testing, large language models, PostgreSQL

1 INTRODUCTION

Imagine running a reporting query on PostgreSQL and getting back a row that should not be there. The query does not crash, no error is raised, and the server log looks normal. Nothing tells you that the answer is wrong. This kind of failure, a *wrong result*, is in our view the most harmful bug a database system can have, because the user has no way to notice it.

Most fuzzers for database systems look for crashes instead. SQLsmith [Seltenreich [n. d.]] generates random queries from the system catalog, and SQUIRREL [Zhong et al. 2020] mutates SQL with coverage feedback; both report a bug when the server crashes or an assertion fails. To catch a wrong result, a fuzzer needs a *test oracle*, something that knows what the right answer is. Differential testing compares two systems on the same query [Slutz 1998]. Metamorphic oracles such as Ternary Logic Partitioning (TLP) [Rigger and Su 2020b], NoREC [Rigger and Su 2020a] and equivalent expression transformation [Jiang and Su 2024] instead compare a query with a rewritten version that must return the same rows. These oracles rewrite the SQL text, so each tool comes with its own small query generator whose output the oracle understands. This leaves a gap. Grammar-based generators reach deep into the planner but do not know which parts of a query can be rewritten safely, while oracle-based tools know how to rewrite but cover only a small part of SQL. Neither of them knows what changed in the code recently, even though the PostgreSQL planner changes every week and most new bugs live in the newest code.

Figure 1 shows a real wrong result that Arbor found. Since PostgreSQL 17, the planner replaces `range @> elem` with the constant `true` when the range is unbounded on both sides, and with `false` when the range is empty. The operator is strict, so for a NULL element the correct answer is NULL. The folded constant gives a wrong answer under `NOT`, in `IS NULL` tests, in `CASE` expressions and in outer joins, where unmatched rows carry NULLs. We reproduced the bug on release 18.6 and on master. Arbor found it with a typed rewrite of the derivation tree, by wrapping the range in a scalar subquery, within the first 150 test cases that used this rewrite. The example is not an isolated case: when we put back 23 recent fixes from PostgreSQL master (Section 5.6), 20 of them changed what a query returns, and only 9 could be seen by a fuzzer that only watches for crashes. The closest previous work is query plan guidance [Ba and Rigger 2023], which steers a generator toward new plan shapes using SQLancer's oracles, and Fuzz4All [Xia et al. 2024], which uses a large language model (LLM) as the generator. Neither connects the generator to the code that changed.

To close this gap we built Arbor, a grammar-aware fuzzer for PostgreSQL based on three ideas. First, a *typed grammar* generates

```
SELECT c,
       '(,)'::int4range @> c AS contains,
         -- must be NULL when c is NULL
       NOT ('empty'::int4range @> c) AS not_empty
         -- must be NULL when c is NULL
FROM (VALUES (NULL::int), (1)) v(c);

  c   | contains | not_empty
------+----------+----------
 NULL | t        | t          ← wrong: the planner folded the call
to a constant
 1    | t        | t
```

Figure 1: A wrong result in PostgreSQL 17, 18.6 and master that Arbor found. The planner support function for range containment returns `true` or `false` for a NULL element, where the strict operator gives NULL. No error or crash reveals it; only an oracle that compares two equivalent queries does.

SQL as derivation trees in which every node knows its type and its place in the query (Section 3.1). Second, the oracles work on the tree rather than on the text: TLP and NoREC can be applied at any query level, alongside typed expression rewrites, a row-level security check, and plan-variant and cross-version oracles (Section 3.2). Third, the grammar is extended from recent commits: an LLM reads a commit and proposes grammar alternatives, seed queries and new equivalence rewrites, and a validation gate accepts them only if the server runs them, they execute the lines the commit changed, and no rewrite changes a result (Section 3.5). Around these ideas, Arbor reduces every finding on its derivation tree and its tokens, localizes a wrong result to a planner path or to a rewrite that is always applied, and drafts the upstream report (Section 3.4).

We evaluated Arbor on PostgreSQL master. The typed grammar reaches 36.7% of backend lines in an hour against 22.2% for random mutation, with 99% valid statements against 26% (**RQ1**), and the oracles double the number of distinct plan shapes. We reported ten bugs found with Arbor, eight of which are fixed upstream, six with our patches, and Arbor rediscovers all ten in one 150-second run (**RQ2**). Of 93 triaged candidates, 8 are real and 15 are duplicates of real bugs, and the false positives come from a small number of classes. On a benchmark of 23 re-injected bugs, no configuration found more than 2 of 20 in one hour (**RQ3**). The LLM extension adds SQL features but no coverage, and the gate accepted one rewrite that is not an identity (**RQ4**). What we learned is that oracle precision, not generator power, limits a wrong-result fuzzer: a generator that reaches more of SQL also reaches more of the corner cases where two "equivalent" queries are in fact not equivalent.

In summary, this paper makes the following contributions:

- a typed PostgreSQL grammar whose derivation trees are used for generation, mutation, metamorphic rewriting and reduction (Section 3);
- tree-based oracles, together with the measured precision of each (Section 5.5);
- a commit-directed LLM extension with a validation gate that tests proposed rewrites on values and on queries (Section 3.5);
- a benchmark of 23 real PostgreSQL bugs put back into one server (Section 5.6); and
- ten reported PostgreSQL bugs, eight of them fixed upstream (six with our patches), released together with the tool and all data (Section 9).

The rest of the paper defines the problem (Section 2), describes the approach and its implementation (Sections 3 and 4), and then reports the evaluation (Section 5).

2 PROBLEM STATEMENT

Before describing Arbor, we fix the terms we use in the rest of the paper. Let G be a typed grammar for the SQL dialect of PostgreSQL and S a database schema. A *test case* is a sequence of statements derived from G over S and run in one transaction that is rolled back at the end. The *derivation tree* $T(Q)$ of a query Q records which alternative each rule took, and its leaves, read from left to right, give the text of Q. A query has a *wrong result* when the server returns, without any error, a multiset $[\![Q]\!]$ that differs from what the SQL standard and the PostgreSQL documentation define. An *oracle* looks at the server's response to a test case and decides between *pass* and *bug candidate*. A metamorphic oracle such as TLP [Rigger and Su 2020b] or NoREC [Rigger and Su 2020a] compares Q with a rewritten query. Similarly, a *plan variant*, the same query run under a setting such as `SET enable_hashjoin = off`, must return the same multiset as the original.

Sound rewrites. A tree rewrite r maps $T(Q)$ to a second tree. We call it *sound* for Q if $[\![Q]\!] = [\![r(Q)]\!]$ holds on every correct server. Some rewrites are sound only under conditions, for example only if no `LIMIT` without `ORDER BY` appears below the rewritten node, and these conditions must be checked on the tree before the rewrite is applied. An unsound rewrite produces *false positives*, that is, candidates that are not bugs.

Goal. Given a fixed time budget B and one server, we want to find as many distinct bugs as possible, where a bug is a crash, a failed assertion, an internal error (SQLSTATE class `XX`) or a wrong result. At the same time, we want the *precision* of each oracle to stay high:

$$\mathit{precision}(o) = \frac{|R_o|}{|R_o| + |F_o|} \tag{1}$$

Here R_o and F_o are the real bugs and the false positives among the distinct candidates reported by oracle o, after de-duplication by signature (Section 3.4). Candidates that do not reproduce (flaky ones) are counted separately. A duplicate of a bug that was already reported upstream is still a real bug, so we also give the precision with duplicates counted in R_o.

Why not let an LLM write the test cases? An LLM can write SQL that targets a specific feature, but it needs a few seconds per call. In an hour it produces at most a few thousand test cases, while the grammar arm produced 71,228 in the same time (Table 1). Queries written by an LLM are also not derivation trees, so the tree-based oracles cannot rewrite them. For these reasons we use the LLM to *extend the grammar* instead: a few calls per commit, after which the grammar generates thousands of variations on its own.

3 APPROACH

Figure 2 gives an overview of Arbor. The central idea is to keep every generated query as a typed derivation tree until it has been

run, so that the oracles can rewrite a single node whose type and position are known, instead of editing SQL text. In Arbor the same tree is used for four things: generation builds it, mutation re-derives one subtree, the oracles rewrite one typed node, and triage shrinks it by replacing subtrees with their smallest derivation. The fuzzing loop (Steps 1–4) runs thousands of test cases per minute, while the LLM loop (Step 5) runs only a few times per commit, before a campaign starts. We describe the steps in order.

3.1 Step 1: Derive Test Cases from a Typed Grammar

We wrote a grammar for the SQL that PostgreSQL 18 and master accept, using a small format of our own (`.pgg`). In the campaigns it had 78 rules with 421 alternatives over 13 SQL types; after the additions for the rediscovery runs (Section 5.5) it has 83 rules and 437 alternatives. A rule can be parameterised by a type, as in `expr<T>`, so a derivation only combines expressions that type-check. Weights decide how often each alternative is chosen. Some things cannot be expressed in a context-free grammar, and for these we use *builtins* written in Python: `@col(T)` picks a visible column of type T in the current scope, `@table` binds a fresh alias, and `@group_keys` turns the current query level into a grouped one, after which only grouped columns and aggregates are visible. The schema has 9 relations, with partitions, inheritance, a view, indexes of four access methods and NULLs in most columns.

A test case consists of up to two setup statements (DDL, DML, `ANALYZE`, or `SET LOCAL` of a planner setting) followed by one or two queries. During derivation, the engine wraps the tokens of every node in invisible markers, so that after rendering it knows the exact token span of each node. Half of the test cases are fresh derivations; the other half take a test case from the corpus and re-derive one of its subtrees. A test case is added to the corpus when it produces a new bug signature, a new SQL feature, a new pair of features, or a new plan shape. A plan shape is a digest of the node types in the `EXPLAIN` output [Ba and Rigger 2023].

Feedback and data. Each alternative has a multiplicative weight that grows when a test case using it enters the corpus and shrinks when its statement fails (rate $\eta = 0.1$, bounded to $[0.2, 5]$). Some literals are taken from a *data profile*, which contains the minimum, maximum and quantiles of every column together with edge values of each type, such as -0, NaN, $\pm$Infinity and the empty array. This way, generated predicates select a real share of the rows. Grammar-based generation itself is not new; what the later steps need from it are the types, the scoping builtins and the node spans.

3.2 Step 2: Attach Oracles as Tree Rewrites

Once a query is derived, Arbor picks at most one metamorphic check for it and stores it with the test case as a pair of queries that must return the same multiset. All of these checks are built on the derivation tree.

Partitioning and counting. *TLP on* `WHERE` chooses a `SELECT` whose result is used as a multiset, which can be the query itself, a derived table, a CTE body, an arm of a set operation, or an `IN`, `EXISTS` or `LATERAL` subquery. It then splits its `WHERE p` into p, `NOT p` and `p IS NULL` and combines the three parts with `UNION ALL`. Since the rewrite edits one node, it also works inside nested queries, where text-based TLP cannot be applied. *TLP on* `HAVING` works the same way, except that the unpartitioned side keeps `HAVING TRUE`: without `GROUP BY`, it is the `HAVING` clause that makes a level a single-group aggregate. *TLP on aggregates* checks that a global `MIN`, `MAX`, `SUM`, `COUNT`, `BIT_*` or `BOOL_*` equals its combination over the three partitions, for example the `MIN` of the `MIN`s or the `SUM` of the `COUNT`s. *NoREC* compares `count(*)` with `WHERE p` against the sum of `CASE WHEN p THEN 1 ELSE 0 END`.

Equivalent forms of one node. Expression rewrites replace one typed expression e with an equivalent form: `COALESCE(e,e)`, `(SELECT e)` or `CASE WHEN TRUE THEN e END` for any type; $e + 0$ or $e \cdot 1$ for numbers; `e || ''` for text; and `NOT NOT p`, `p AND TRUE` or `p OR FALSE` for predicates. *Predicate rewrites* turn `x IN (a, b)` into `(x = a) OR (x = b)`, which PostgreSQL 18 converts back into `= ANY`; they also turn `a < b` into `NOT (a >= b)`, which three-valued logic keeps exact, and `BETWEEN` into two comparisons. The *NOT NULL conjunct* rewrite turns `WHERE p` into `WHERE p AND c IS NOT NULL` for a NOT NULL column c that p uses, to exercise the planner's NOT NULL simplifications; it is skipped under outer joins and for generated columns, where such a column can still be NULL. Finally, *ORDER BY removal* drops the statement's own `ORDER BY`, and *CTE materialisation* toggles `MATERIALIZED`. The *table fence* turns one table reference into `(SELECT * FROM t OFFSET 0)`, which prevents the planner from pushing predicates into it, removing it, or pruning its partitions.

Security and estimates. For *row-level security*, five tables have a policy, defined in one file that both the server and the rewrite use. The query runs once as a restricted role, under the policies, and once as a superuser with every protected table t replaced by `(SELECT * FROM t WHERE policy)`. If the two results differ, the policies either let a row through or hide a row they should not. Following CERT [Ba and Rigger 2024], adding a `WHERE` clause to a query must not increase the planner's row estimate; this check compares the `EXPLAIN` estimates, and a violation is reported as a *performance* candidate, separately from bugs.

In every case, a check is attached only if its rewrite is sound for that query (Section 2).

3.3 Step 3: Run the Test Case Several Ways

The server runs each test case inside a transaction, with one savepoint per statement, and rolls it back at the end, so test cases cannot affect each other. For each deterministic query, Arbor runs three extra things: (i) both sides of the attached check; (ii) the query under two of 18 *plan variants*, chosen at random per query, such as hash join only, no hash aggregation, forced parallel plans, partitionwise join, no eager aggregation, or JIT; and (iii) the whole test case on a server of the last release, PostgreSQL 18.6, with a separate database for each worker (the *cross-version oracle*). Results are compared as order-insensitive digests. The digests ignore differences that no plan controls: trailing zeros of equal `numeric` values, the sign of a float zero, the last digits of finite floats in aggregates, and the text of equal `interval` values.

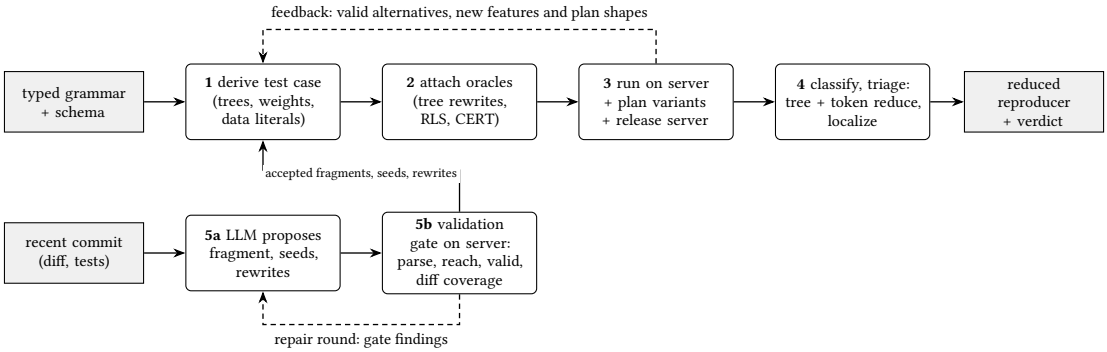


Figure 2: Overview of Arbor. The top row is the fuzzing loop: test cases are derived from the typed grammar, oracles are attached as rewrites of the derivation tree, and the server runs each query under several settings. The bottom row runs before a campaign: an LLM turns a recent commit into grammar alternatives, and the validation gate accepts them only when the server runs them and they reach the changed lines.

3.4 Step 4: Classify, De-duplicate, Triage

The oracle looks at the statement results and at the server log. A `TRAP` line means an assertion failure, a lost connection without one means a crash, an error of class `XX` is an internal error, and a confirmed mismatch of any digest is a wrong result. A metamorphic mismatch counts only if it repeats when both sides are run again, and crash evidence counts only if it carries the process id of the test case's own backend or its parallel workers. Each candidate gets a *signature*, which is the assertion location, the normalised backtrace or error message, or, for wrong results, the oracle and the shape of the query. Only the first test case with a given signature is saved.

Triage then reproduces the candidate three times on a fresh connection and reduces it in two stages. *Tree reduction* works on the derivation trees that the grammar arm stores with every failure. It drops a statement, drops one repetition, or replaces a subtree with the smallest derivation of the same rule and type, and replays the trees through the generator, which re-binds names, aliases and types. As a result, every candidate is valid SQL, and the server's time is spent only on checking whether the failure is still there. *Token reduction* comes next: delta debugging [Zeller and Hildebrandt 2002] removes statements, then clauses, list items and parenthesised groups level by level over the parenthesis tree [Misherghi and Su 2006], and finally single tokens. A step is kept only if the same failure still occurs, meaning the same assertion, the same normalised message, or the same oracle and plan variant. For a metamorphic candidate, the two queries are reduced together so that they stay a valid pair.

Localization. For a reduced wrong result, Arbor runs the queries under every plan variant. If some variant removes the disagreement, the bug is in the code path that this setting switches, and the report names the setting and its source files. If no variant helps, the cause is a rewrite that is always applied, constant folding, a planner support function, or expression evaluation. The output of triage is a short reproducer, the rows each side returned, the localization and a draft report in the format of the PostgreSQL bug form.

3.5 Step 5: Extend the Grammar from Recent Commits

The last step connects the grammar to the code that changed. It has two parts.

Step 5a: propose. Arbor lists the commits of PostgreSQL master that change the optimiser, the executor, the parser or SQL-callable functions and that add regression tests. For each commit it builds a digest containing the commit message, the changed functions, the SQL the commit added to the tests, and the *diff-coverage targets*, which are the changed lines of each C file. The LLM receives this digest together with a fixed, cached system prompt that describes the `.pgg` format, the builtins, the conventions of the grammar, the schema and the whole current grammar. It answers with structured output: a rationale, a fragment of new alternatives, one to five seed test cases, the features it targets, and up to three *equivalence rewrites*. An equivalence rewrite is a template over one expression of a given type, such as `CASE WHEN {e} IS NULL THEN NULL ELSE {e} END`, that the model claims equals `{e}` for every value. The commit miner never shows the model a fix that the benchmark of Section 5.6 reverts.

Step 5b: validate. No model output reaches a campaign unless the server agrees with it. The validation gate runs five checks in order: (i) *parse*: the fragment parses with the grammar, adds no types and uses only known builtins; (ii) *reachability*: in a sample of 150 test cases with the new alternatives boosted, they are used at least five times; (iii) *validity*: at least 60% of the statements that use them run without error; (iv) *seeds*: each seed runs; and (v) *diff coverage*: on a gcov build, the sample executes more of the commit's changed lines than the same sample without the fragment. When a check fails, its findings (the parse error, the most frequent error messages, the uncovered lines) go back to the model for up to three repair rounds. Every round is recorded in a ledger with its tokens, cost and the gate's report.

A proposed rewrite is a claim about semantics, so the gate tests it twice. First, the server evaluates `rewrite(v) IS NOT DISTINCT FROM v` for NULL, the edge values of the type and quantiles of the

data. Then Arbor applies the rewrite alone to the expressions of 150 test cases, with confirmation re-runs. The rewrite is accepted only if no value and no query shows a difference, after which it becomes one of the expression rewrites of Step 2. Neither test varies the collation, so the gate also rejects any template with an explicit `COLLATE` (Section 5.7).

4 IMPLEMENTATION

We implemented Arbor in Python. It is about 11,000 lines of code with 151 unit tests, plus the grammar, which had 628 lines in the campaigns and has 651 after the additions described in Section 5.5. Arbor talks to PostgreSQL through `psycopg` and reads the server log to find evidence of assertion failures and crashes. Every trial runs as a resumable process that writes its events, timeline and failures as JSON, so an interrupted trial can continue where it stopped. The code is split into six parts:

- the `.pgg` parser and derivation engine, which handles scopes, type variables, node spans and the `MINIMAL` marker used by reduction;
- the generator, with the corpus, the alternative weights and the data profile;
- the executor, which manages savepoints and plan variants and runs the metamorphic, RLS and CERT checks, with a wrapper for the cross-version oracle;
- triage, covering reproduction, tree and token reduction, localization, report drafts, bisection and the precision ledger;
- the benchmark scripts, including `bench verify` and `bench attribute`; and
- the LLM arm, made of the commit miner, the prompts, a client that stores every response so that a run can be replayed, the validation gate and the ledger.

A single configuration file selects the generator, the oracles and the budget of a campaign, so every arm in the evaluation differs only in its configuration.

5 EVALUATION

We evaluated Arbor with four research questions:

RQ1 Does the typed grammar reach more of PostgreSQL than a random-mutation generator in the same time, in terms of valid statements, covered lines and branches, SQL features and plan shapes?
RQ2 Which bugs does Arbor find, and how precise is each oracle?
RQ3 When real bugs are put back into PostgreSQL, which configuration finds how many of them, and how fast?
RQ4 What does the commit-directed LLM extension add, and what does it cost in tokens, money and repair rounds?

5.1 Subject and Benchmark

All servers are built from PostgreSQL master `26bd08fb` (16 October 2026) with assertions enabled. For **RQ3** we built a benchmark in the style of Magma [Hazimeh et al. 2020] by reverting fixes. We took every commit between 31 July and 16 October 2026 that fixes a bug in the optimiser, executor, parser, rewriter or a SQL-callable function, adds a regression test, and reverts cleanly both alone and together with the others; this gave 23 commits. From them we build the *fixed* server, one *solo* server per bug with only that fix reverted, and the *injected* server with all 23 fixes reverted, on which the campaigns run. For each bug we wrote a proof of concept (PoC) from its regression test. The PoC changes its result for 20 of the bugs: 8 wrong results, 5 internal errors, 4 assertion failures, 2 missing errors and 1 wrong error. A fuzzer that only detects crashes can see just the 9 assertions and internal errors among them. The remaining three bugs change only the plan, or need an equality operator that returns NULL for non-NULL inputs, so we leave them out of the denominator. On the injected server, every PoC shows the same symptom as on its solo server.

5.2 Configurations

All configurations (we call them *arms*) share the server, schema, executor, oracles, budget and per-trial seeds, and differ only in the generator. Random mutates seed queries at the token level and attaches no metamorphic checks. Grammar is Step 1 alone. Grammar+Oracles adds Steps 2–4: metamorphic checks (including RLS and CERT) on half of the queries, plan-shape guidance, feedback weights and data-profile literals. Grammar+Oracles, flat is the same arm without the feedback weights. +LLM adds the fragments, seeds and rewrites that passed the validation gate, while +LLM fragments and +LLM seeds use only one of these. The commit miner excludes the 23 benchmark fixes. Every arm uses the plan-variant oracle, and in **RQ3** also the cross-version oracle.

5.3 Setup

Each arm runs 5 one-hour trials with four workers, each worker with its own server. **RQ1**, **RQ2** and **RQ4** ran on one laptop (WSL2, 24 logical cores) and **RQ3** on another, so each experiment is compared on a single machine. The cross-version reference is PostgreSQL 18.6. Following Klees et al. [Klees et al. 2018] and Arcuri and Briand [Arcuri and Briand 2011], we report medians, compare arms with the two-sided Mann–Whitney U test, and give the Vargha–Delaney $\hat{A}_{12}$ effect size [Vargha and Delaney 2000]. With 5 trials per arm the smallest possible two-sided p is 0.008, so we read the effect sizes and not only the p-values. Coverage is sampled every 300 seconds per worker, which is why the first point of every curve is the coverage of the server after start-up.

The language model. We used the client's manual mode, in which a script answered the prompts with Google Gemini Flash models on the free tier (9 answers by `gemini-3.6-flash`, 2 by `gemini-3.8-flash` and 14 by `gemini-3-flash-preview`). The daily quota limited **RQ4** to 10 of the 25 mined commits, and the results of **RQ4** hold for these models.

5.4 RQ1: Reach

Table 1 gives the medians of the seven arms, and Figure 3 shows how coverage and plan shapes grow over time.

Observation 1. The typed grammar reaches far more than random mutation: 36.7% against 22.2% of lines ($\hat{A}_{12} = 1.00$, $p = 0.008$), 101 against 23 SQL features, and 99.3% against 26.0% valid statements, although the random arm runs 15 times more test cases.

Table 1: RQ1: median of 5 one-hour trials per arm on an instrumented PostgreSQL master. Valid is the share of statements that run without error; lines and branches are of the backend (gcov); features are of a catalogue of 148.

Arm	Valid (%)	Lines (%)	Branches (%)	Features	Plan shapes	Test cases
Random	26.0	22.2	15.1	23	43	1,074,240
Grammar	99.3	36.7	27.1	101	20,279	71,228
Grammar+Oracles	96.3	36.2	26.7	101	34,313	58,601
Grammar+Oracles, flat	99.4	36.2	26.8	101	23,997	64,923
+LLM fragments	95.5	35.7	26.5	104	25,972	48,222
+LLM seeds	98.3	35.6	26.4	106	29,274	63,655
+LLM (both)	95.3	35.7	26.4	106	26,375	46,754

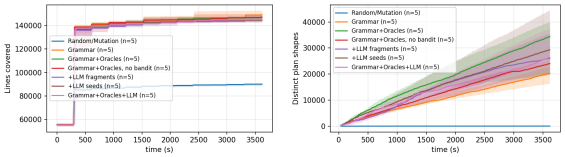


Figure 3: RQ1: lines covered (left) and distinct plan shapes (right) over time, median and interquartile band of 5 trials per arm.

Observation 2. The oracles do not add line coverage (36.7% to 36.2%, $p = 0.42$), but they roughly double the number of plan shapes (20,279 to 34,313, $\hat{A}_{12} = 0.84$, $p = 0.095$; with 5 trials this is an effect size, not a significant difference). The feedback weights contribute to this: without them the same arm reaches 23,997 plan shapes ($\hat{A}_{12} = 0.80$, $p = 0.15$), but its validity is higher (99.4% against 96.3%, $p = 0.016$).

The oracles are not free, however. The plan-variant runs and the metamorphic checks lower the throughput from 67 to 40 statements per second.

5.5 RQ2: Bugs and Oracle Precision

Across the **RQ1** and **RQ3** campaigns, together with the earlier hunt and smoke runs, Arbor reported 93 distinct bug candidates after de-duplication by signature. We triaged all of them as described in Section 3.4; Table 2 gives the verdicts per oracle.

Bugs. Table 3 lists the ten bugs we reported. Six of them came from running the fuzzer, either in the campaigns or in later runs on newer master builds. The other four came from a targeted bug hunt that we did alongside the campaigns, in which we audited recent planner and executor features with differential probes. Eight of the ten bugs are now fixed upstream, six of them with fixes written by us, and the remaining two are open with our patches. The oracles also found three bugs that others had reported a few days before us, which we count as duplicates: a wrong result of self-join elimination under `ROLLUP` in 18.6 (#19800, found by a plan variant), #19772 (found by the `(SELECT e)` rewrite) and #19784.

Rediscovery. Since four of the bugs came from the hunt and not from the fuzzer itself, we wanted to know whether Arbor can find all of them. We added each reproducer as a seed that carries its own check (for a wrong result, the query that must agree with it). We also extended the grammar with what the four hunt bugs need: statistics on expressions, statistics import with boundary values, a user-defined implicit cast, and `IGNORE NULLS` over whole-partition frames. In addition, we added a *hang* oracle that restarts a server whose statement runs ten seconds past its `statement_timeout`. After these changes, one 150-second trial of Grammar+Oracles on the base build reported all nine bugs reported up to that point, each matched to its catalogue entry, with unchanged validity (99.3%). When we later added the tenth reproducer (#19826, found by the plan-variant oracle under partitionwise aggregation), the one-command reproduction, a single 150-second trial that replays the seeds in turn, reported all ten bugs.

Observation 3. Plan-variant and metamorphic oracles found real wrong results in released versions that no crash oracle can see (BUG #19793 and #19800).

Precision. Of the 93 candidates, 28 (30%) did not reproduce in three re-runs. Of the other 65, 8 are real, 15 are duplicates of real bugs and 42 are false positives, which gives a precision of $8/50 = 0.16$ without duplicates and $23/65 = 0.35$ if duplicates count as true positives. The false positives come from only a few classes. Six classes were fixed before the campaigns. The five classes that the campaigns revealed were fixed *after* them; we tested these fixes on the stored reproducers but did not re-measure them in a new campaign. The five classes are: equal `interval` values printed differently depending on join order (16 candidates; the digest now uses PostgreSQL's own interval equality); order-sensitive aggregates sorted by a key that is not the value, such as `ORDER BY NULL` (2); state that diverges in the cross-version oracle after an `INSERT` that only master accepts (5); an RLS rewrite that left a protected table inside a repeated sub-select unfiltered (5); and an LLM rewrite that is not an identity (Section 5.7). One candidate that remained after these fixes turned out to be a new bug, which we reported as BUG #19826: `corr()` of a constant input is NULL when computed serially but 1 under partial aggregation. It is now fixed upstream with our patch (`ef1b7404`, back-patched to 19).

5.6 RQ3: Re-injected Bugs

Observation 4. With one hour and five trials, the arms found few of the benchmark bugs: Random 0 of 20, Grammar 1, Grammar+Oracles 2 and +LLM 2. No arm found any of the 8 wrong-result bugs.

Table 2: RQ2: verdict of every triaged candidate per oracle. Duplicate: a real bug that had already been reported upstream. False positives include one intended change between versions. Flaky: did not reproduce in three re-runs. Open: not decided yet.

Oracle	Candidates	Real	Duplicate	False pos.	Flaky	Open
assertion	15	3	10	1	1	0
crash	1	0	0	1	0	0
cross-version	25	0	0	7	18	0
internal_error	4	2	1	0	1	0
mm: expr	3	1	1	1	0	0
mm: rls	5	0	0	5	0	0
mm: tlp_having	4	0	0	4	0	0
mm: tlp_where	4	1	1	2	0	0
plan-variant	32	1	2	21	8	0
all	93	8	15	42	28	0

Table 3: The ten bugs we reported upstream, the oracle that recognises each, and their state on 13 November 2026. All reproduce on the base build `26bd08fb`, which predates the fixes.

Bug	Report	Oracle	State
`JSON_TABLE` assigns no collations	#19768	assertion	fixed (our patch)
`range @> NULL` folded to a constant	#19793	`(SELECT e)` rewrite	reported, patch
`MERGE`: SubPlan with no parent	#19802	internal error	fixed
parallel `Append` assertion	#19803	TLP (+LLM fragments)	fixed
`to_char(NaN)` breaks `NORMALIZE`	#19771	assertion	fixed (our patch)
`NOT IN` anti-join ignores a cast	pgsql-bugs	TLP	fixed (our patch)
`IGNORE NULLS` frame never returns	pgsql-bugs	hang	fixed (our patch)
duplicate statistics expression	pgsql-bugs	assertion	fixed (our patch)
non-finite imported MCV frequencies	pgsql-hackers	CERT estimate	patch
`corr()` of a constant under partial aggregation	#19826	plan variant	fixed (our patch)

Table 4: RQ3: trials (of 5) in which an arm found each re-injected bug, with the median time to the first finding. The rows below list, per symptom, how many bugs an arm found at all. Only bugs found by some arm have their own row; 18 of 20 were found by none.

Bug	Symptom	Random	Grammar	Grammar+Oracles	+LLM (both)
`11b56ee2`	assertion	0/5	0/5	1/5 (2901 s)	1/5 (1858 s)
`dfcf6fa7`	internal error	0/5	5/5 (458 s)	4/5 (509 s)	5/5 (1109 s)
wrong result, 8 bugs: found		0	0	0	0
internal error, 5 bugs: found		0	1	1	1
assertion, 4 bugs: found		0	0	1	1
missing error, 2 bugs: found		0	0	0	0
wrong error, 1 bug: found		0	0	0	0
Bugs found (of 20)		0	1	2	2

The grammar arms found the internal error of fix `dfcf6fa7` in nearly every trial (median 458, 509 and 1,109 seconds), while Random never did. The two arms with oracles also found the assertion of `11b56ee2`, once each, which is too little to separate them. So the benchmark does *not* show an advantage of the oracles; that claim rests on **RQ2**. We see three reasons for this result. First, one hour is short for bugs that need a specific combination of features. Second, our attribution is conservative: of the 30 wrong-result failures, 11 reproduce on the fixed server, 19 on no server, and none on a solo server. Third, five trials cannot separate arms that find a bug in only one trial out of five.

5.7 RQ4: The LLM Extension

The gate accepted 4 of the 10 commits, 2 in the first round and 2 after repair. Of the 25 rounds, 16 failed because too few statements ran (the mean validity of a proposal was 36.5%), 4 because the fragment was never derived or did not parse, and 1 on the seeds or on coverage. The whole run used 328,000 input and 171,000 output tokens, at no cost on the free tier.

Observation 5. The LLM arms add SQL features (101 to 104–106, $\hat{A}_{12} = 1.00$, $p = 0.004$) but no coverage: lines drop from 36.2% to 35.7% ($p = 0.42$), plan shapes from 34,313 to 26,375, and validity from 96.3% to 95.3%.

Over the 10 commits, the accepted fragments cover only 34 more changed lines. Still, the fragments-only arm found BUG #19803, so the benefit with these models is real but modest.

The gate is not enough for rewrites. All 7 proposed rewrites passed the gate, but one of them, `NULLIF({e}, NULL::text COLLATE "POSIX")`, is not an identity: it changes the collation of its argument, and it caused a false positive in **RQ3**. Neither the value probe nor the live checks vary the collation, which is how it got through. The gate now rejects any explicit `COLLATE`, but the general point remains: testing an equivalence on samples does not prove it.

6 DISCUSSION

The results raise a few questions that we want to discuss before turning to the threats to validity.

Is a larger grammar always better? Not for a wrong-result fuzzer. A larger grammar reaches more corner cases of SQL semantics, and some of them break the assumptions an oracle makes: `HAVING` without `GROUP BY`, `DISTINCT` over `numeric` values of different scale, or `interval` values that are equal but print differently. The price is paid in oracle precision, and every rewrite needs a soundness check on the tree.

Why keep the tree? Mutation re-derives a subtree, the oracles rewrite one typed node, and reduction replaces a subtree with its smallest derivation. In each case the generator replays the tree and re-binds names and types, so the result stays valid SQL. On the `JSON_TABLE` bug, tree reduction followed by token reduction gave a 79-token reproducer, whereas token reduction alone stopped at 116 tokens. On the tree, a clause is one node with a known span, which lets Arbor apply TLP inside a derived table or a `LATERAL` subquery, where a text-based rewrite would have to parse the query again.

Why compare with the last release? Plan variants cannot catch a wrong result that every plan shares, such as a constant that is folded before any plan is chosen (Figure 1). The last release does not have the newest planner rules, so it disagrees exactly on the bugs that new code introduces. It also disagrees on intended changes, which triage has to recognise.

Can an LLM be trusted? Not on its word. A rewrite is a claim about semantics, and **RQ4** shows that our gate accepted a wrong one. For fragments the situation is easier, because the model only has to name SQL that reaches a commit, and the validation gate rejects anything that does not parse, is never derived, fails on the server or misses the changed lines. Repair rounds helped only rarely: of the 4 accepted commits, 2 passed in the first round, and 6 of the 10 commits never passed.

Does the approach carry over to other engines? After the PostgreSQL campaigns we wanted to know whether the same design works beyond PostgreSQL, so we generalized Arbor to DuckDB, SQLite, Turso and Apache DataFusion (branch `generalized` of our repository). The answer is yes, with a small adapter per engine. Each engine runs in its own worker process, and a dialect *overlay* of a few dozen lines drops or rewrites the parts of the PostgreSQL grammar that the engine does not accept. With hand-written overlays, 96.3% of the generated queries are valid on DuckDB, 97.7% on SQLite, 93.3% on Turso and 79.4% on DataFusion. An overlay miner can also draft the overlay from the measured failure rate of each grammar alternative: one mined round lifted DuckDB from 32.1% to 44.1% valid statements without any line written by hand. Instead of the PostgreSQL plan variants, Arbor uses optimizer-rule firing, read from rule switches or from per-rule plan traces, as feedback and as a coverage measure. We also added rows synthesized with Z3 that make TLP partitions non-empty, and normalizers learned from probes for cross-engine comparison; between DuckDB and SQLite, these normalizers reduce held-out disagreements from 28.7% to 3.6%. The engines' own test suites serve as seeds, and a loop triages new failures and drafts a filing package with a regression test in the engine's own format. Finally, *rule kernels*, minimal queries that each make one optimizer rule fire, are composed in pairs. In 15-minute campaigns (medians of three trials per arm), they raise the number of distinct optimizer-rule pairs reached from 307 to 386 on DuckDB, from 241 to 276 on DataFusion, and from 86 to 182 on SQLite. Every kernel trial beat every plain one ($\hat{A}_{12} = 1.00$), although three trials per arm cannot give $p < 0.1$.

These campaigns found fourteen bugs. Nine of them were already fixed on the engine's development branch when we checked. For the other five we filed an issue and a pull request with a fix and a regression test: Turso #11594 / PR #11595 (`percentile_disc` with an expression argument) and #11796 / PR #11797 (an aggregate in `IN (SELECT ...)` next to a window function, which gives a spurious error or a panic); DuckDB #28551 / PR #28552 (`min`/`max` over a nested value computed by a subquery, a 2.0.0 regression), which a maintainer approved for merging and for the 2.0 release line; DataFusion #27235 / PR #27236 (`UPDATE` and `DELETE` statements that repeat an expression fail on an internal common-subexpression column); and a DataFusion internal error for a scalar subquery inside `VALUES`. A single command (`scripts/reproduce_generalized.sh`) reproduces all five on the released versions that the campaigns ran on.

7 THREATS TO VALIDITY

Internal validity. We tried to make the arms comparable: every arm runs 5 trials with the same per-trial seeds on the same build, schema, executor, oracles and budget; each trial has its own server; a metamorphic mismatch counts only if it repeats; and every candidate is reproduced three times before triage. Still, five trials are few. Differences with p above 0.008 are effect sizes, not significant results, and several of our conclusions rest on them.

Our own oracles can also be wrong. Eleven false-positive classes reached triage, six before the campaigns and five during them (Section 5.5). We fixed the last five *after* the campaigns and checked the fixes on the stored reproducers and with unit tests. We did not re-run the campaigns, so the precision after the fixes is not a measurement, and more classes may remain. The verdicts are our own, and some were made on the reduced reproducer; we release all reproducers so that others can check them. An accepted LLM rewrite is tested, not proven (Section 5.7). Attribution in **RQ3** replays each finding on single-bug servers, so a finding that needs two injected bugs at once is reported as unattributed.

The two laptops differ, so **RQ1** and **RQ3** cannot be compared in absolute throughput. In addition, 14 **RQ1** trials were re-run with the same seeds after a server failed crash recovery.

External validity. We test one system at one commit, with one schema of 9 relations and one set of row-level security policies that we chose ourselves. Bugs that need other data shapes, extensions or concurrent sessions are out of reach. The 23 benchmark bugs are recent fixes with regression tests, which may favour features that recent PostgreSQL development touched. The LLM results hold for free-tier Gemini Flash models on 10 commits; we store every prompt and response so that the run can be replayed.

Construct validity. Coverage measures which code ran, not whether an oracle could have noticed a bug there. A bug "found" in **RQ3** means that a test case failed on its solo server and passed on the fixed one, even if its symptom differs from the PoC's. A fragment that passes the validation gate reaches the changed lines, but that does not prove it tests the changed behaviour. Finally, the rediscovery of the reported bugs replays their reproducers as seeds: it shows that the oracles recognise each bug, not that generation alone would find it in a short run.

8 RELATED WORK

Metamorphic oracles for database systems. The work closest to ours is by Rigger and Su: pivoted query synthesis [Rigger and Su 2020c], NoREC [Rigger and Su 2020a] and TLP [Rigger and Su 2020b], all implemented in SQLancer, together with equivalent expression transformation by Jiang and Su [Jiang and Su 2024]. These tools have found hundreds of bugs, but they build the follow-up query from strings produced by their own small generators. We use the same kinds of oracles, but apply them to typed derivation trees of a grammar that covers much more of PostgreSQL, at any query level, and we report the precision of each oracle on this larger language.

Guidance for database fuzzing. Query plan guidance [Ba and Rigger 2023] steers SQLancer toward new plan shapes, and we use plan shapes in a similar way as a corpus signal. SQUIRREL [Zhong et al. 2020] mutates an intermediate representation of SQL with coverage feedback to find crashes, and SQLRight [Liang et al. 2022] adds the logic-bug oracles of SQLancer to this setting. CERT [Ba and Rigger 2024] checks that a more restrictive query never gets a larger cardinality estimate, and we include its check. Directed greybox fuzzing [Böhme et al. 2017] steers a fuzzer toward chosen code locations. We also direct the campaign toward recent commits, but at the level of SQL features rather than branches.

Grammar-based and differential SQL testing. RAGS [Slutz 1998] compared the results of several database systems on random SQL, and SQLsmith [Seltenreich [n. d.]] generates queries from the catalog of a running PostgreSQL. Instead of comparing different systems, we compare one server with itself under 18 plan variants and with its own last release, which avoids most dialect differences. We need a typed grammar for this because the oracles must know the type and the scope of every node.

LLMs for fuzzing. Fuzz4All [Xia et al. 2024] uses an LLM directly as the input generator. We use the LLM differently: we call it only a few times per commit, for grammar alternatives and rewrites, and everything it proposes has to pass a server-side validation gate.

Benchmarks, evaluation and reduction. Magma [Hazimeh et al. 2020] ports real bugs into real programs, and Klees et al. [Klees et al. 2018] showed how fuzzing evaluations go wrong without repeated trials. Following them, our benchmark reverts real fixes in one server, attributes each finding by replaying it on single-bug servers, and runs every arm for repeated trials. We reduce failures with delta debugging [Zeller and Hildebrandt 2002] and hierarchical delta debugging [Misherghi and Su 2006], and we add paired reduction for metamorphic checks, where the two queries must shrink together to stay equivalent.

9 CONCLUSION

We presented Arbor, a fuzzer that generates SQL as typed derivation trees, rewrites these trees into metamorphic checks, compares each query with its plan variants and with the last release, and extends its grammar from recent commits behind a server-side gate. With it we reported ten PostgreSQL bugs, eight of which are fixed upstream, six with our patches. Its grammar reaches 36.7% of backend lines against 22.2% for random mutation, yet in the end it was the precision of the oracles, not the reach of the generator, that decided how useful the findings were. We think this is the main thing to improve in future wrong-result fuzzers: each false-positive class we found came from an oracle assumption that held for most queries but not for all, so every new rewrite and every comparison needs its own soundness argument. Finally, with a dialect overlay and a small adapter per engine, the same core tests DuckDB, SQLite, Turso and DataFusion, where it found five more bugs that we reported upstream with fixes (Section 6).

DATA AVAILABILITY

Our replication package is the course repository https://github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi (MIT license); the generalized fuzzer of Section 6 is on branch `generalized` of the same repository. The `README.md` lists the exact dependencies and links every upstream report with its state. It also gives a one-command reproduction of the headline result (`scripts/reproduce.sh`, which finds the ten reported PostgreSQL bugs) and one for the five bugs of the generalized fuzzer (`scripts/reproduce_generalized.sh`). The repository contains the source of Arbor with its unit tests, the grammar and the schema, and the benchmark with the 23 fix commits, their PoCs and the scripts that build the fixed, solo and injected servers. It also holds the experiment configurations, the logs and reports of every trial, the triage verdicts, and the LLM ledger with every prompt and response, so that **RQ4** can be replayed without calling the model.

USE OF GENERATIVE AI

Arbor calls a large language model as part of its method (Section 3.5). In the evaluation this was Google Gemini Flash (Section 5); every prompt and response, together with the model that served it, is in the replication package. An AI coding assistant helped with parts of the implementation and with drafting text. The authors designed the approach, reviewed all code and text, and checked every statement, citation and number. All numbers reported here were produced by running the tool and the scripts.

STATEMENT OF CONTRIBUTIONS

Every member of the team can explain every part of the project. The split below records who did most of the work on each part, and the commit history of the repository shows the same split (GitHub users `v28k`, `Blazing-bhuvi` and `doma2542`).

Venu Kummari. Designed and implemented Arbor: the typed grammar and its generator, the metamorphic oracles on derivation trees, the plan-variant, cross-version, row-level security and cardinality oracles, the executor and the hang oracle, triage with tree and token reduction, the LLM gate, the re-injected bug benchmark and the evaluation harness. Reported BUG #19768 (`JSON_TABLE`) and BUG #19793 (`range @> NULL`) with patches, and BUG #19826 (`corr()` under partial aggregation), which was fixed upstream with his patch. Wrote the paper. Built the generalized fuzzer for DuckDB, SQLite, Turso and DataFusion (Section 6) and wrote its five upstream fixes.

Bhuvan Chilukuti. Ran the **RQ3** campaign on the re-injected bug benchmark and its attribution, and the cross-version, metamorphic and audit-driven bug hunts on recent PostgreSQL features. Reported, with patches, the `NOT IN` anti-join bug, the `IGNORE NULLS` hang, the duplicate statistics expression assertion and the non-finite MCV frequencies, and answered the maintainers on these threads.

Gowtham Reddy. Built the servers, ran the **RQ1**, **RQ2** and **RQ4** campaigns (35 trials) and wrote their reports, ran the LLM arm with Gemini Flash over ten commits, and triaged the campaign candidates. Reported BUG #19802 (`MERGE`) and BUG #19803 (parallel `Append`), both fixed upstream, as reporter and tester, and reported BUG #19771 (`to_char`) and wrote its fix, which was committed upstream.

Signed, 13 November 2026: Venu Kummari Bhuvan Chilukuti Gowtham Reddy

REFERENCES

Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. 1–10. https://doi.org/10.1145/1985793.1985795

Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2060–2071. DOI: verify.

Jinsheng Ba and Manuel Rigger. 2024. CERT: Finding Performance Issues in Database Systems Through the Lens of Cardinality Estimation. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 133:1–133:13.

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3133956.3134020

Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A Ground-Truth Fuzzing Benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3, Article 49 (2020). https://doi.org/10.1145/3428334

Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 821–835.

George T. Klees, Andrew Ruef, Benjamin Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3243734.3243804

Yu Liang, Song Liu, and Hong Hu. 2022. Detecting Logical Bugs of DBMS with Coverage-based Guidance. In *31st USENIX Security Symposium (USENIX Security)*. 4309–4326.

Ghassan Misherghi and Zhendong Su. 2006. HDD: Hierarchical Delta Debugging. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*. 142–151.

Manuel Rigger and Zhendong Su. 2020a. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/3368089.3409710

Manuel Rigger and Zhendong Su. 2020b. Finding Bugs in Database Systems via Query Partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). https://doi.org/10.1145/3428279

Manuel Rigger and Zhendong Su. 2020c. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 667–682.

Andreas Seltenreich. [n. d.]. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed: 2026-10-23.

Donald R. Slutz. 1998. Massive Stochastic Testing of SQL. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*. 618–622.

András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. https://doi.org/10.3102/10769986025002101

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3597503.3639121

Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200.

Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 955–970.